

Chapter 18

멀티 스레드 기반의 서버 구현

목차

- 스레드 개요
 - 스레드와 프로세스 차이점
 - 스레드 관련 함수: 스레드 생성/종료 함수
 - Worker thread
- 공유 자원 동기화 문제점
- 스레드 동기화 기법
 - Mutex
 - Semaphore
- 스레드 종료 대기 및 소멸
- 멀티 스레드 기반의 다중 접속 서버

멀티 프로세스의 문제점 및 스레드 등장 배경

1. 프로세스 자원 소모

생성 비용 발생

프로세스 생성에는 많은 리소스가 소모됨

Context Switching 오버헤드

다수의 프로세스 실행 시 빈번한 프로세스간 전환으로 시스템 성능 저하가 발생

- 현재 상태 저장 및 복원
- 하드디스크 액세스 수반

2. 데이터 교환의 어려움

독립적 메모리 구조

각 프로세스는 독립된 메모리를 사용하여 직접 공유가 불가능

IPC 기법의 필요성

운영체제의 지원이 필요하며 구현이 복잡

- Pipe, Message Queue
- Shared Memory 등

3. 스레드 사용

경량화된 프로세스

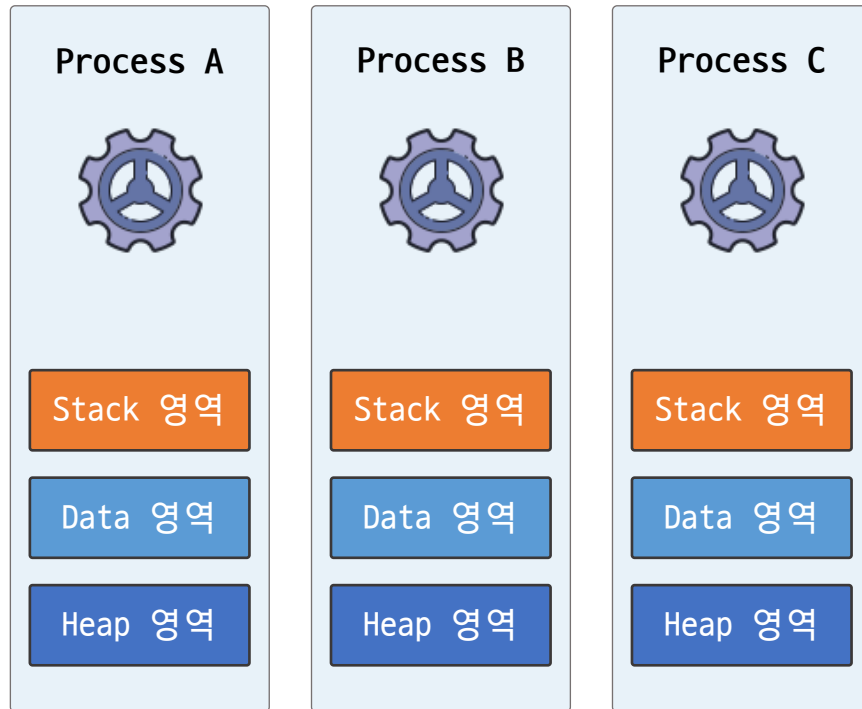
프로세스보다 가벼운 구조로 빠른 Context Switching이 가능

효율적인 데이터 공유

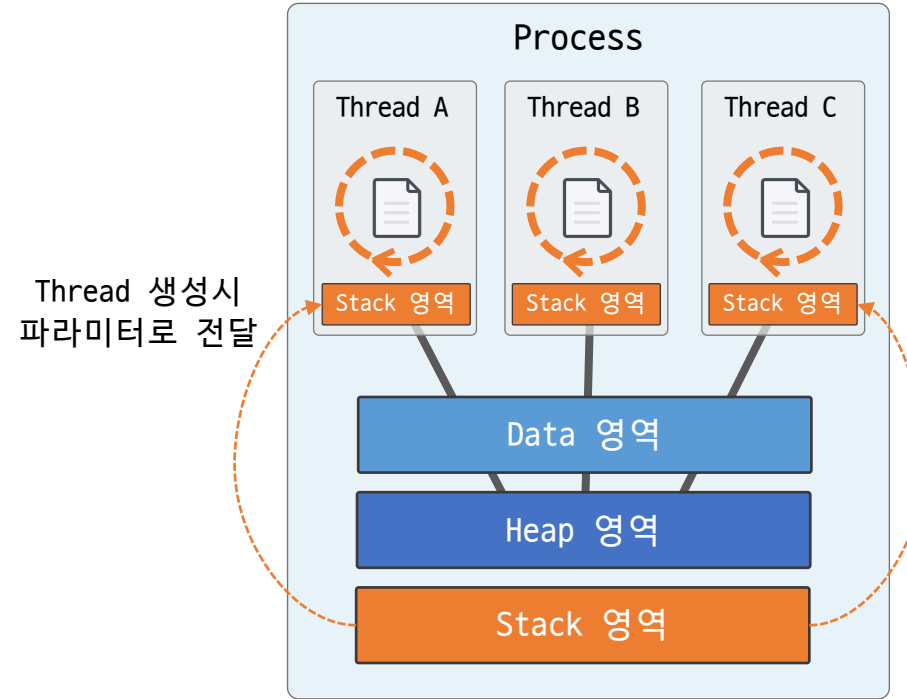
메모리 영역을 공유하므로 별도의 IPC 없이 통신 가능합니다.

- 응답성 향상

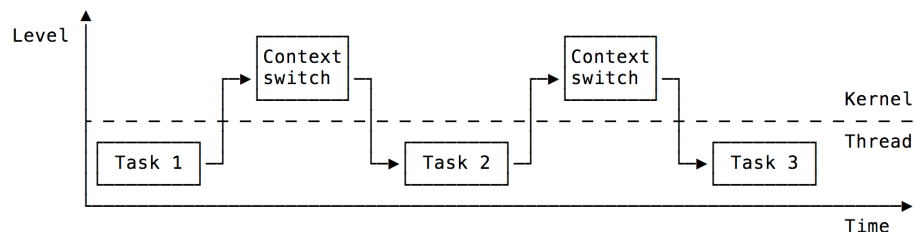
프로세스와 스레드의 차이점



- 프로세스는 완전히 독립적으로 동작
- 독립된 메모리 공간을 가짐



- 스레드
 - 하나의 프로세스 내부에서 동작
 - 스레드는 Data 영역과 Heap 영역을 공유함
 - 공유하는 메모리 영역으로 인해 스레드간 데이터 교환이 쉽게 이루어짐



스레드 생성 함수

■ pthread_create() 함수

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *restrict thread, const pthread_attr_t *restrict attr,  
                  void *(*start_routine)(void*), void *restrict arg);
```

-> 성공 시 0, 실패 시 0 이외의 값 반환

pthread_t: unsigned long 타입

restrict 키워드

- 특정 메모리에 접근할 수 있는 포인터가 단 한 개라는 의미의 제약
- memcpy(void *restrict dst, const void *restrict src, size_t n);

➤ thread

- 생성할 스레드의 id를 저장하기 위한 변수의 주소 전달: unsigned long 타입
- 스레드는 프로세스와 마찬가지로 스레드 구분을 위해 id가 부여됨

➤ attr

- 스레드에 부여할 특성 정보
- NULL: 기본적인 특성의 스레드가 생성

➤ start_routine

- 함수 포인터(스레드의 main 함수 역할을 수행하는 함수의 주소 전달):
- void * 반환 및 void * 하나를 인자로 받음

➤ arg

- 스레드 함수에 전달할 인자의 주소(NULL 가능)

사용 예: pthread_create(&t_id, NULL, thread_main, (void*)&thread_param)

매개 변수로 전달

thread1.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* thread_main(void *arg);
int main(int argc, char* argv[])
{
    pthread_t t_id;
    int thread_param = 5;

    if(pthread_create(&t_id, NULL, thread_main,
                    (void*)&thread_param) != 0)
    {
        puts("pthread_create() error");
        return -1;
    }

    sleep(10);
    puts("end of main");
    return 0;
}
```

- 프로세스의 종료를 막기 위해 sleep(10) 호출
- 10초간 main() 함수의 실행을 지연시킴

5

```
void* thread_main(void *arg)
{
    int i;
    int cnt = *((int*)arg);

    for(i=0; i < cnt; i++)
    {
        sleep(1);
        puts("running thread");
    }
    return NULL;
}
```

- 5회 반복 (arg=5)
- 1초 지연 x 5회 = 총 5초 지연

- lpthread 옵션
- . 스레드 라이브러리의 링크를 지시

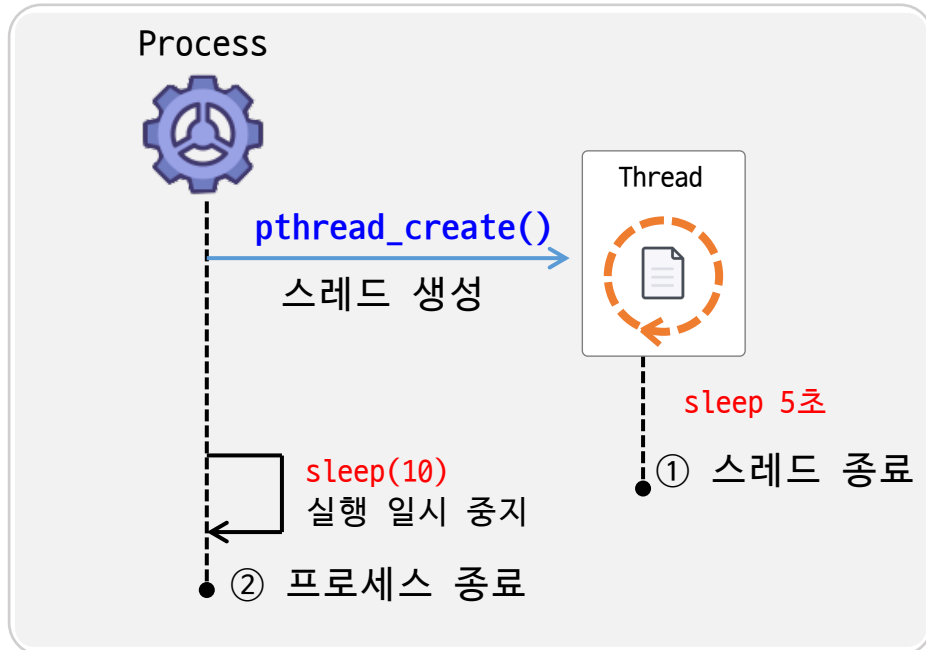
```
$ gcc thread1.c -o tr1 -lpthread
$ ./tr1
running thread
running thread
running thread
running thread
running thread
end of main
```

5초 소요

- 스레드 함수 thread_main()은 스레드 생성과 동시에 실행
- 스레드가 생성되면 스레드는 등록된 스레드 함수를 실행
- 스레드 함수의 실행이 완료되면 스레드는 종료

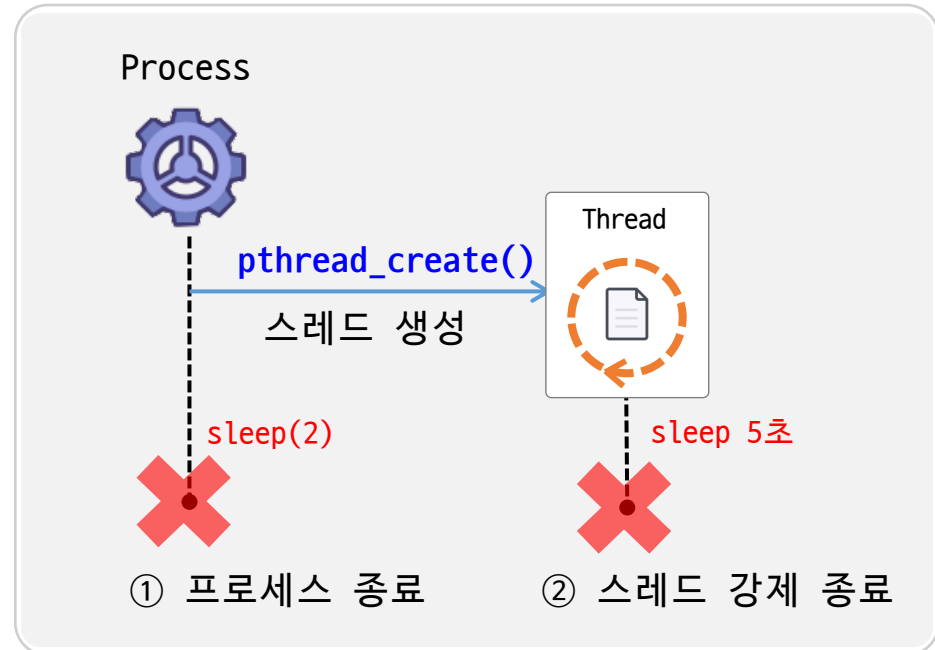
프로세스와 스레드 종료

스레드 정상 종료



- 단순히 `sleep()` 함수를 이용해 프로그램의 흐름을 임의로 지연시키는 방식에는 한계가 있음

스레드 강제 종료



- 프로세스가 종료되면 그 내부에서 생성된 스레드도 역시 자원을 반납하고 함께 소멸됨

`sleep(10)`을 `sleep(2)`로 수정하면, thread가 "running thread"를 5회 출력하지 못하고 종료됨

스레드의 종료 대기

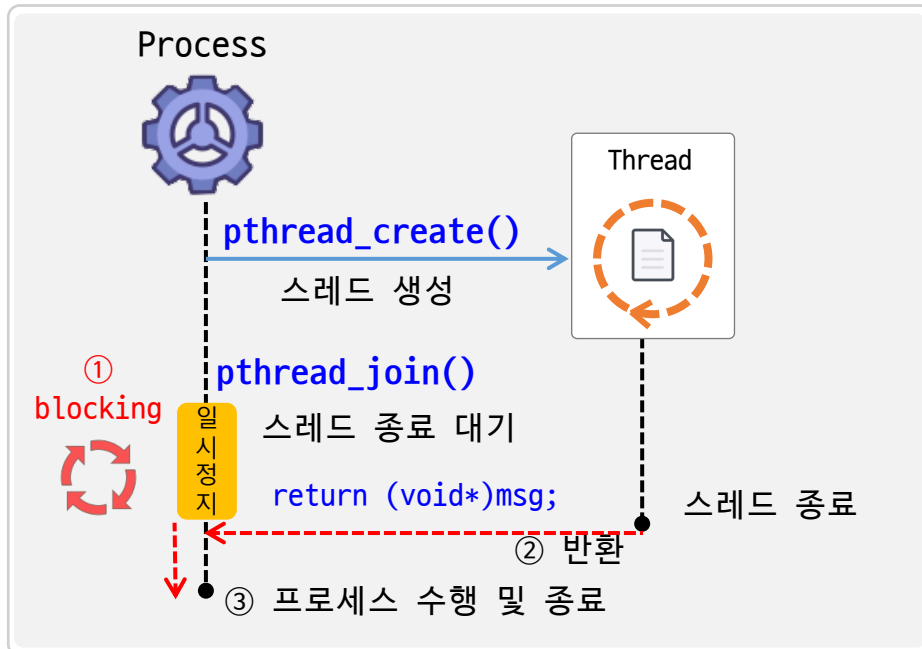
■ pthread_join() 함수

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **status);
```

-> 성공 시 0, 실패 시 0 이외의 값 반환

- thread: 종료를 기다릴 스레드의 id
- status: 스레드 함수가 반환하는 값(void *)이 저장될 포인터 변수의 주소
 - pthread_join() 함수를 통해 종료된 스레드의 반환 값을 전달받을 수 있음



- pthread_join()을 호출한 프로세스(또는 스레드)는 스레드가 종료될 때까지 blocking 상태가 됨

pthread_join() 함수 호출의 예: thread2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
```

```
void* thread_main(void *arg);
//char msg[50] = {0}; // 전역 변수 사용
```

```
int main(int argc, char* argv[])
{
```

```
    pthread_t t_id;
    int thread_param = 5;
    void *thr_ret;
```

```
    if(pthread_create(&t_id, NULL, thread_main, (void*)&thread_param) != 0)
    {
        puts("pthread_create() error");
        return -1;
    }
```

```
    if(pthread_join(t_id, &thr_ret) != 0)
    {
        puts("pthread_join() error");
        return -1;
    }
```

```
    printf("Thread return message: %s\n", (char*)thr_ret);
    free(thr_ret);
    return 0;
}
```

Heap 영역 대신에 지역 변수 사용시
warning 발생

공유메모리

Heap 영역

char *msg =
malloc(50);
"Hello, I am thread~"

thread가 반환하는 메모리 주소

thread_main이
종료될 때까지 대기

스레드에서 생성한 Heap 영역(msg)을
main()에서 free(thr_ret)

```
void* thread_main(void *arg)
```

```
{
```

```
    int i;
```

```
    int cnt = *((int*)arg);
```

```
    // Heap 영역 사용 (프로세스와 스레드 사이에 데이터 공유)
```

```
    char *msg = (char*)malloc(sizeof(char) * 50);
```

```
    //char msg[50] = {0}; // stack: warning 발생
```

```
    strcpy(msg, "Hello, I am thread~ \n");
```

```
    for(i=0; i < cnt; i++)
```

```
    {
```

```
        sleep(1);
```

```
        puts("running thread");
```

```
    }
```

```
    return (void*)msg;
```

```
}
```

Heap 사용 권장하지 않음
(전역 변수 사용 권장)

Heap 영역의 메모리 주소 전달

실행 결과

```
$ gcc thread2.c -o thread2 -lpthread
```

```
$ ./thread2
```

```
running thread
```

```
running thread
```

```
running thread
```

```
running thread
```

```
running thread
```

```
Thread return message: Hello, I am thread~
```

스레드에 안전한 함수

■ 스레드에 안전한 함수, 불안정한 함수

- 스레드에 안전한 함수 (Thread-safe function)
 - 둘 이상의 스레드가 동시에 호출해도 문제가 발생하지 않는 함수
- 스레드에 불안정한 함수 (Thread-unsafe function)
 - 둘 이상의 스레드가 동시에 호출할 경우 문제가 발생하는 함수

• 스레드에 안전한 함수의 예

```
struct hostent* gethostbyname(const char *hostname);
```

불안전

```
struct hostent* gethostbyname_r(const char *name, struct hostent *result,  
                                char *buffer, int buflen, int *h_errnop);
```

안전

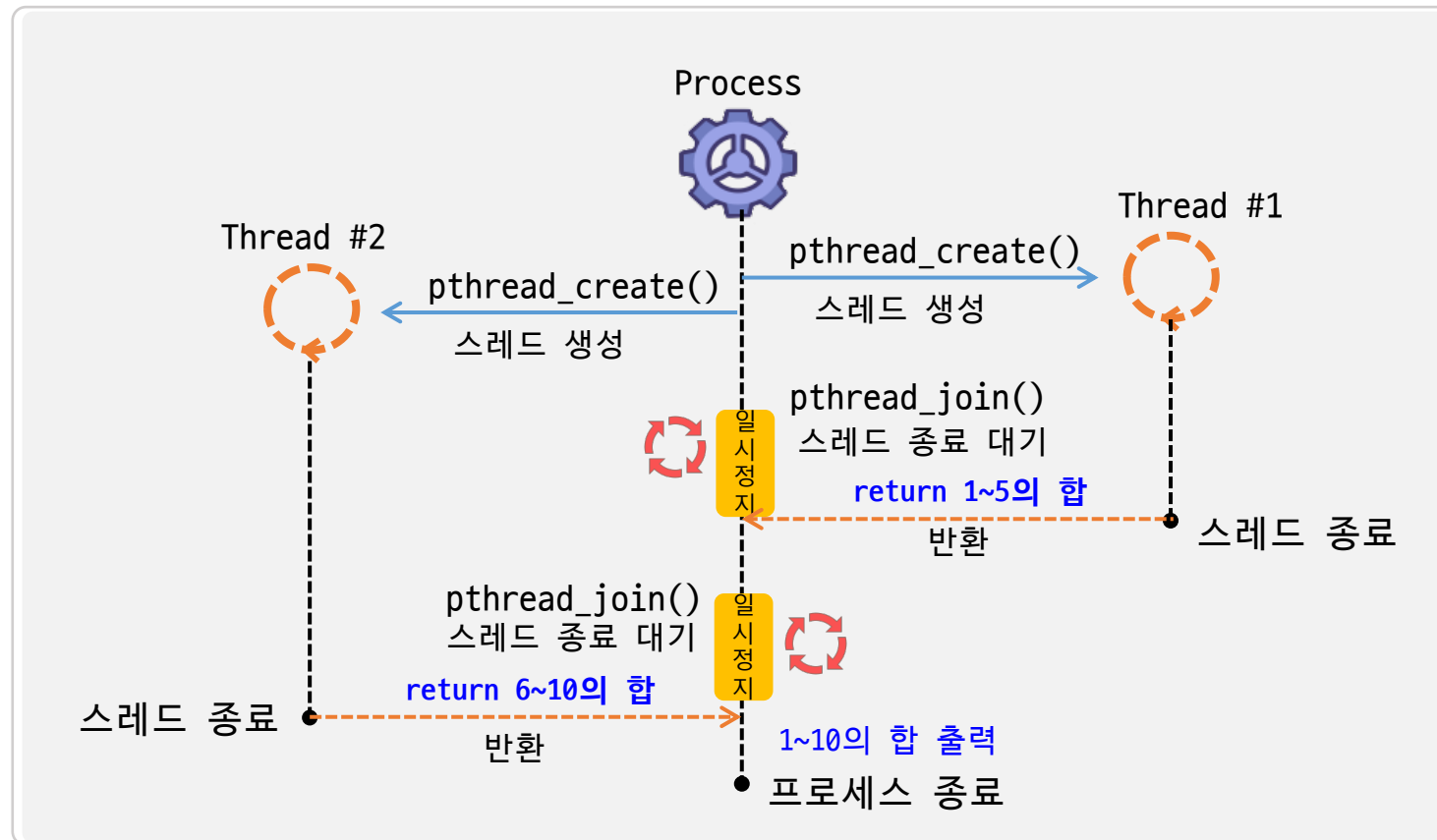
```
$ gcc -D_REENTRANT mythread.c -o mthread -lpthread
```

- 컴파일 옵션에 **-D_REENTRANT** 추가
 - 스레드에 불안정한 함수 호출 코드를 스레드에 안전한 함수 호출로 자동 변경하여 컴파일

Worker Thread 모델

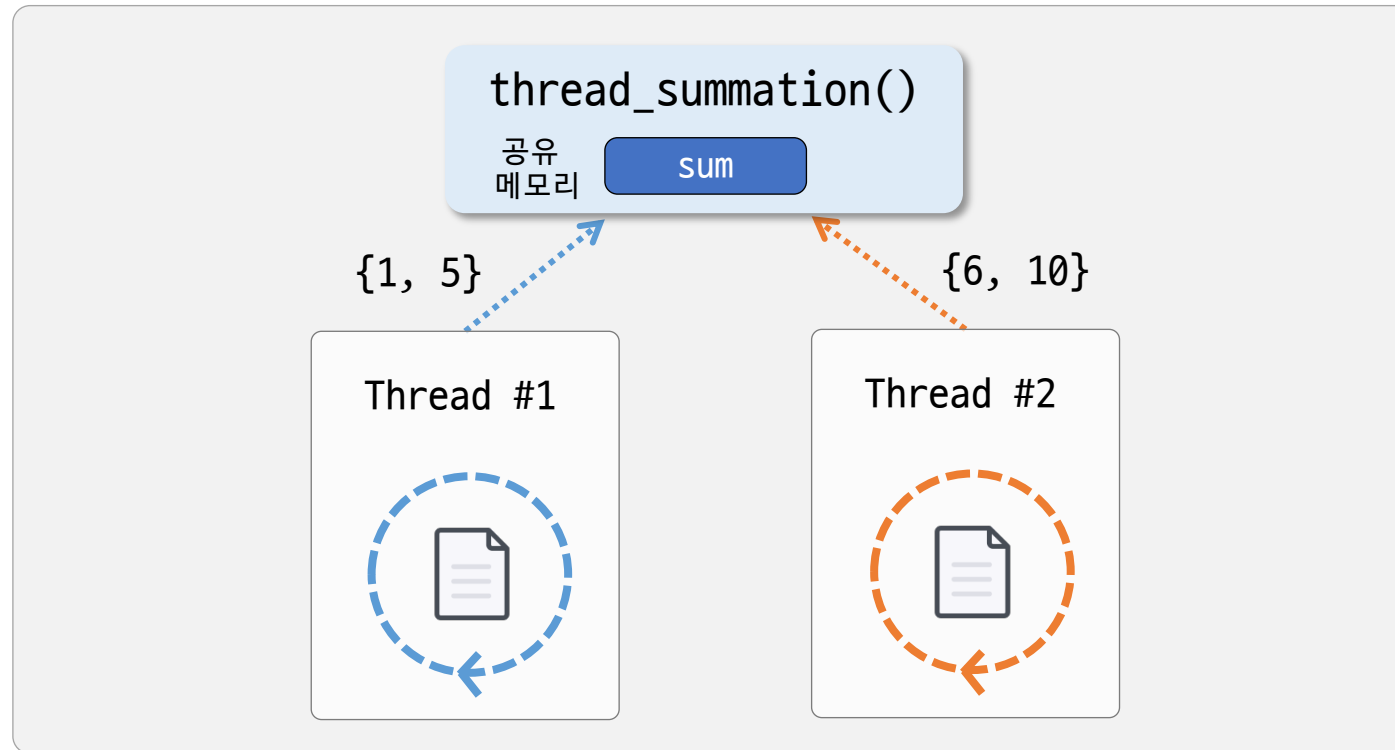
■ Worker Thread 모델

- 멀티 스레드(Multithread) 환경의 대표적인 디자인 패턴
- 여러 스레드(Worker)에게 작업을 분담하여 처리하게 한 후,
- 메인 스레드가 그 결과를 취합하는 모델



워커(Worker) 스레드 모델의 예

- 여러 스레드의 공유 메모리(전역 변수) 동시 접근 예시
 - 스레드 간의 충돌(동기화 문제) 발생



- 둘 이상의 스레드가 전역변수 `sum`에 동시에 접근
 - 동기화 문제가 발생하여, `sum` 변수에 예상치 못한 잘못된 결과값이 저장될 수 있음

Worker Thread 모델 (thread3.c)

```
#include <stdio.h>
#include <pthread.h>
```

```
void *thread_summation(void *arg);
```

```
int sum = 0;
```

전역변수: 공유 메모리

```
int main(int argc, char *argv[])
```

```
{
```

```
    pthread_t id_t1, id_t2;
```

```
    int range1[] = {1, 5};
```

```
    int range2[] = {6, 10};
```

```
    pthread_create(&id_t1, NULL, thread_summation, (void*)range1);
```

```
    pthread_create(&id_t2, NULL, thread_summation, (void*)range2);
```

```
    pthread_join(id_t1, NULL);
```

```
    pthread_join(id_t2, NULL);
```

```
    printf("result: %d\n", sum);
```

```
    return 0;
```

```
}
```

```
void* thread_summation(void *arg)
```

```
{
```

```
    int start = ((int*)arg)[0];
```

```
    int end = ((int*)arg)[1];
```

```
    while(start <= end)
```

```
    {
```

```
        sum += start;
```

```
        start++;
```

```
    }
```

```
    return NULL;
```

```
}
```

- 두 스레드가 전역 변수 sum에 동시 접근

- 2개의 스레드가 thread_summation() 함수를 호출

Thread #1



start = 1
end = 5

Thread #2



start = 6
end = 10

실행 결과

```
$ gcc thread3.c -D_REENTRANT -o tr3 -lpthread
```

```
$ ./tr3
```

```
result: 55
```

공유 자원 동기화 문제: thread4.c (실습)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>
#define NUM_THREAD 100
```

```
void * thread_inc(void * arg);
void * thread_des(void * arg);
long long num=0;
```

공유 자원

```
int main(int argc, char *argv[])
{
```

```
    pthread_t thread_id[NUM_THREAD];
    int i;
```

```
    for(i=0; i<NUM_THREAD; i++)
    {
```

```
        if(i%2==1)
```

```
            pthread_create(&(thread_id[i]), NULL, thread_inc, NULL);
```

```
        else
```

```
            pthread_create(&(thread_id[i]), NULL, thread_des, NULL);
```

```
    }
```

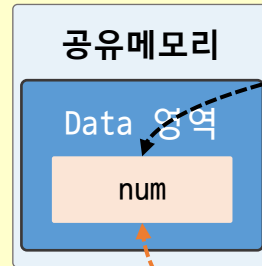
```
    for(i=0; i<NUM_THREAD; i++)
```

```
        pthread_join(thread_id[i], NULL);
```

```
    printf("result: %lld \n", num);
```

```
    return 0;
```

```
}
```



- 50개의 thread: thread_inc() 실행
- 50개의 thread: thread_des() 실행

```
void * thread_inc(void * arg)
{
    int i;
    for(i=0; i<50000000; i++)
        num += 1;

    return NULL;
}
```

```
void * thread_des(void * arg)
{
    int i;
    for(i=0; i<50000000; i++)
        num -= 1;

    return NULL;
}
```

```
$ gcc thread4.c -D_REENTRANT -o tr4 -lpthread
$ ./tr4
result: 30264
```

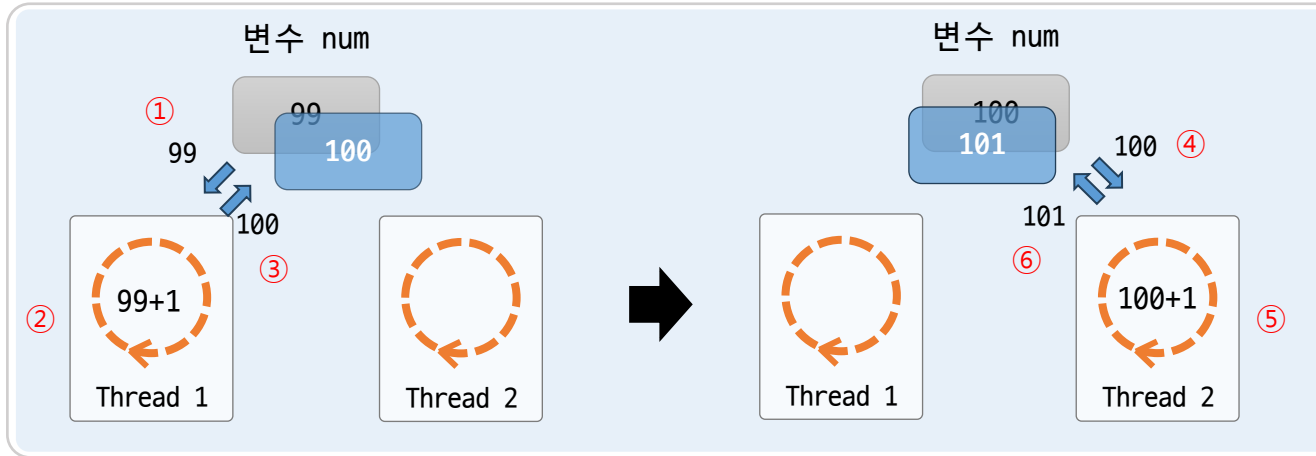
```
$ ./tr4
result: 2920844
```

실행할 때마다
결과값이 달라짐

- 두 스레드가 공유 자원(num)을 동시에 접근하여
 - 스레드 1: num += 1 을 수행
 - 스레드 2: num -= 1 을 수행

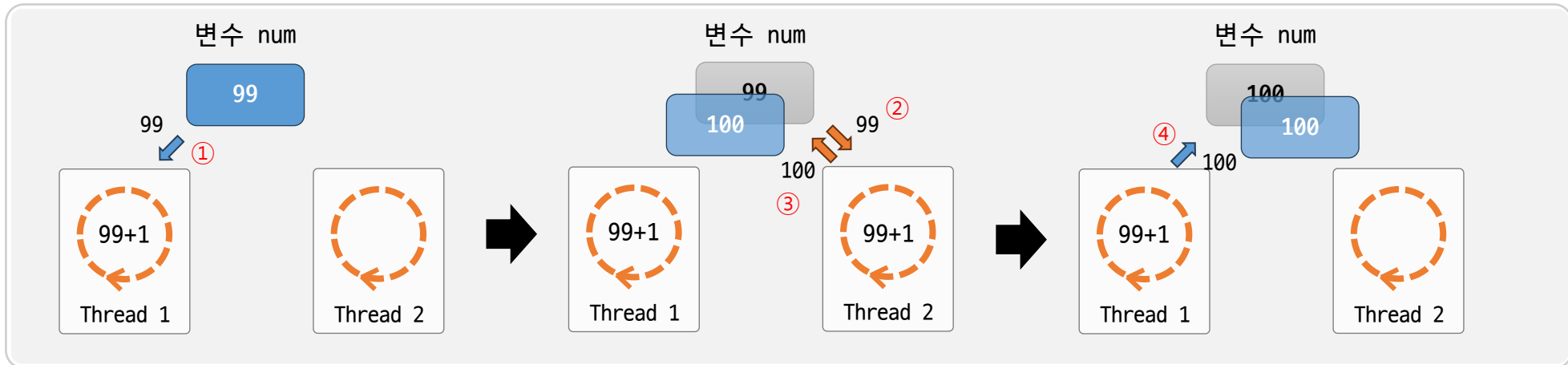
두 스레드의 공유 자원 동시 접근 문제점

이상적인
상황



순차적으로 변수 `num`에 접근하면
문제가 발생하지 않는다.

잘못된
접근의 예



- thread1과 thread2가 각각 1씩 증가시켰는데, 변수 `num`의 값은 1만 증가됨
- 하나의 스레드가 변수 `num`에 접근해서 연산을 하는 동안, 다른 스레드가 변수 `num`에 접근하지 못하도록 막아야 함: 동기화(Synchronization)

어셈블러 코드 확인

■ C 코드를 어셈블리어로 변환

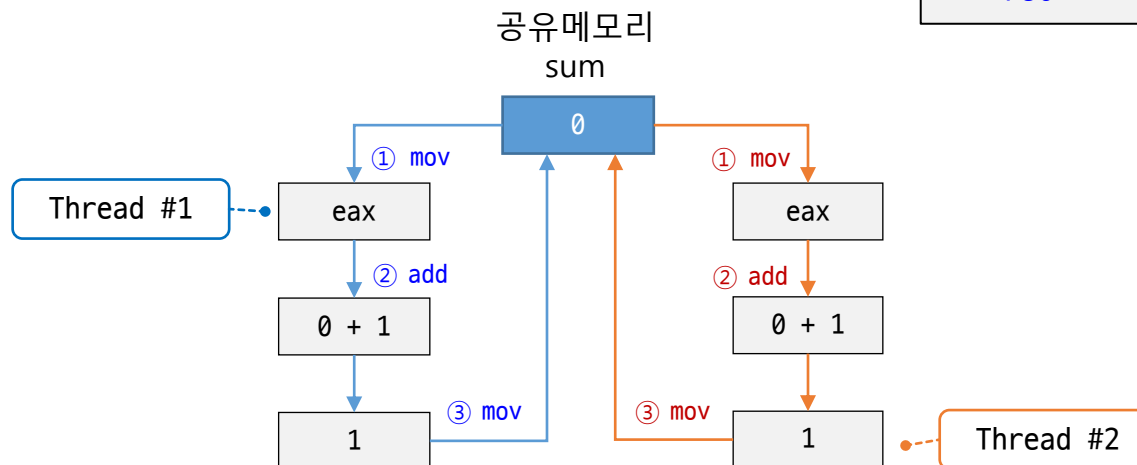
- <https://gcc.godbolt.org>

```
int sum=0;
int main()
{
    sum += 1;
    return 0;
}
```

- rbp (Stack Base Pointer): 스택의 시작 주소
- rsp (Stack Pointer): 스택 복귀 주소
- eax (Extended Accumulator Register): 산술 연산값을 저장
- rip (Instruction Pointer) 레지스터: 현재 명령의 위치를 가리키는 레지스터

```
sum:
    .zero 4
main:
    push rbp
    mov rbp, rsp
    mov eax, DWORD PTR sum[rip] ①
    add eax, 1 ②
    mov DWORD PTR sum[rip], eax ③
    mov eax, 0
    pop rbp
    ret
```

sum += 1 연산
- 최소 3단계
- 각 단계에 다른 thread가
진입 가능



어셈블러 코드 확인: objdump

```
#include <stdio.h>
```

```
int sum=0;
```

```
int main()
```

```
{
```

```
    sum += 1;
```

```
    return 0;
```

```
}
```

```
$ gcc -c sum.c
```

```
$ objdump -d sum.o
```

```
sum.o:      file format elf64-x86-64
```

```
Disassembly of section .text:
```

```
0000000000000000 <main>:
```

0:	f3 0f 1e fa	endbr64	
4:	55	push	%rbp
5:	48 89 e5	mov	%rsp,%rbp
8:	8b 05 00 00 00 00	mov	0x0(%rip),%eax # e <main+0xe>
e:	83 c0 01	add	\$0x1,%eax
11:	89 05 00 00 00 00	mov	%eax,0x0(%rip) # 17 <main+0x17>
17:	b8 00 00 00 00	mov	\$0x0,%eax
1c:	5d	pop	%rbp
1d:	c3	ret	

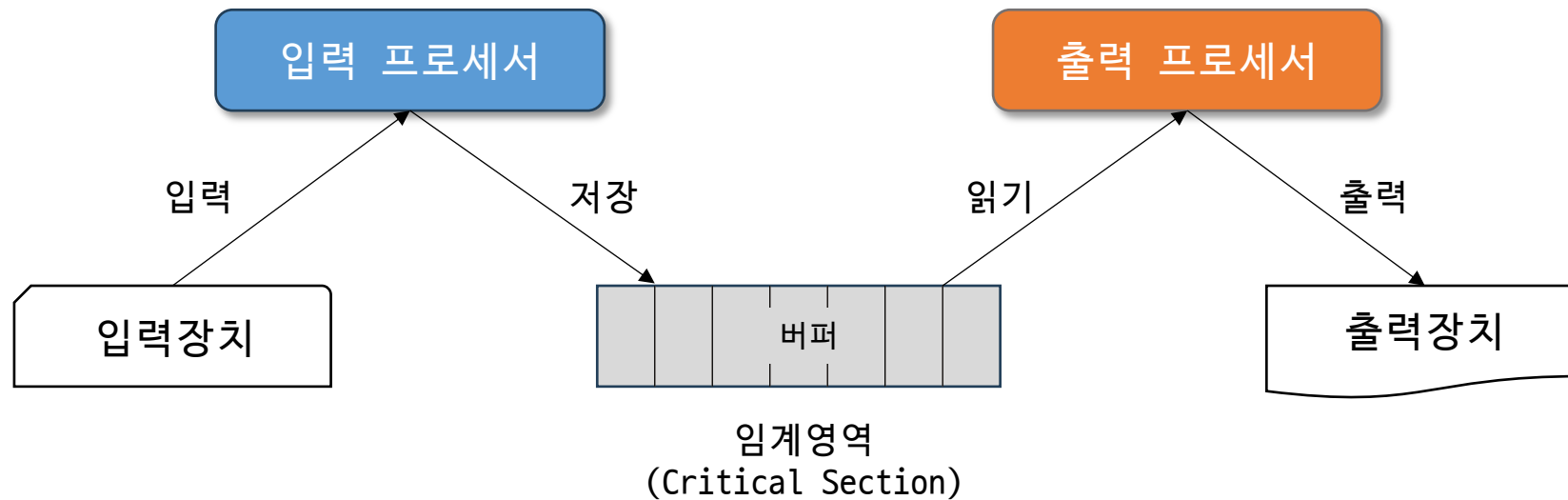
- rbp (Stack Base Pointer): 스택의 시작 주소
- rsp (Stack Pointer): 스택 복귀 주소
- eax (Extended Accumulator Register): 산술 연산값을 저장
- rip (Instruction Pointer) 레지스터: 현재 명령의 위치를 가리키는 레지스터

임계 영역 (Critical Section)

- 임계 영역의 개념

- 둘 이상의 스레드가 동시에 접근하여 실행할 경우 문제가 발생할 수 있는 코드 영역
- 한 번에 오직 하나의 프로세스(또는 스레드)만 진입하여 사용할 수 있는 영역
 - 공유 자원의 독점권을 보장해야 하는 코드 영역

- 임계 영역의 예



임계 영역은?

두 스레드 함수

```
void * thread_inc(void * arg)
{
    int i;
    for(i=0; i<500000000; i++)
        num += 1;    // 임계영역

    return NULL;
}

void * thread_des(void * arg)
{
    int i;
    for(i=0; i<500000000; i++)
        num -= 1;    // 임계영역

    return NULL;
}
```

- 서로 다른 함수 내의 코드라도, 동시에 실행될 경우 문제가 발생할 수 있음
 - 두 스레드가 동일한 메모리 공간(공유 메모리)에 동시 접근하기 때문

스레드 동기화 (Thread Synchronization)

■ 동기화가 필요한 대표적인 상황

- 여러 스레드가 동일한 메모리 영역(공유 자원)에 동시에 접근하는 상황
- 스레드 간의 명확한 실행 순서 제어가 필요한 상황

– 동시 접근을 차단(상호 배제)하거나, 스레드 간의 실행 순서를 지정하여 메모리 일관성을 유지

■ 대표적인 동기화 기법

- 뮤텝스(Mutex) 기반 동기화
- 세마포어(Semaphore) 기반 동기화

– 운영체제 차원에서 제공하는 기능
– 운영체제마다 지원 기법 및 적용 방법이 다를 수 있음

Semaphore:

A system of sending messages(signals) using two flags(신호 장치)



뮤텍스(Mutual Exclusion: 상호 배제)

- 뮤텍스(Mutex) 개념
 - 여러 스레드가 공유 자원에 동시 접근하는 것을 차단하는 동기화 기법
 - 오직 한 번에 하나의 스레드만 공유 자원(임계 영역)에 접근 가능
- 뮤텍스 동작 과정
 - 공유 자원 사용 전 Lock을 획득(acquire)한 다음에 공유 자원을 사용: `pthread_mutex_lock()`
 - Lock을 얻지 못한 스레드는 열쇠가 반환될 때까지 대기(blocking)
 - 공유 자원 사용 완료 후 Lock 반환(release): `pthread_mutex_unlock()`



무텍스 기반의 동기화

■ 무텍스의 생성과 소멸

```
#include <pthread.h>
```

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

-> 성공 시 0, 실패 시 0 이외의 값 반환

➤ mutex

- 무텍스 생성시 무텍스의 참조 값 저장을 위한 변수의 주소 전달
- 무텍스 소멸 시에는 소멸하고자 하는 무텍스의 값 전달

➤ attr: 생성하는 무텍스의 특성 정보를 담고 있는 변수(NULL)

```
pthread_mutex_t mutex;
```

```
pthread_mutex_init(&mutex, NULL);
```

```
    . . .
```

```
pthread_mutex_destroy(&mutex);
```

무텍스 기반의 동기화 기본 구성

- 무텍스의 획득과 반환

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
-> 성공 시 0, 실패 시 0 이외의 값 반환
```

- 무텍스 기반 동기화의 기본 구성

```
pthread_mutex_lock(&mutex);
// 임계 영역의 시작
. . .
// 임계 영역의 끝
pthread_mutex_unlock(&mutex);
```

무텍스 기반의 동기화 예제: mutex.c #1 (실습)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>
#define NUM_THREAD 100

void* thread_inc(void *arg);
void* thread_des(void *arg);
long long num = 0;
pthread_mutex_t mutex;

int main(int argc, char *argv[])
{
    pthread_t thread_id[NUM_THREAD];
    int i;
    pthread_mutex_init(&mutex, NULL);
    for(i=0; i<NUM_THREAD; i++)
    {
        if(i%2)
            pthread_create(&(thread_id[i]), NULL, thread_inc, NULL);
        else
            pthread_create(&(thread_id[i]), NULL, thread_des, NULL);
    }
    for(i=0; i<NUM_THREAD; i++)
        pthread_join(thread_id[i], NULL);
    printf("result: %lld\n", num);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

전역 변수로 선언

Mutex 생성

Mutex 삭제

mutex.c #2 (실습)

```
void *thread_inc(void *arg)
{
    int i;
    pthread_mutex_lock(&mutex);
    for(i=0; i<50000000; i++)
        num += 1;
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

임계 영역을
넓게 설정

```
void *thread_des(void *arg)
{
    int i;
    for(i=0; i<50000000; i++)
    {
        pthread_mutex_lock(&mutex);
        num -= 1;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

임계 영역을
좁게 설정

- 임계영역을 넓게 설정한 경우: thread_inc()
 - 뮤텝스의 lock/unlock 함수 호출 횟수를 최소화하여 **실행 속도가 빠름**
 - 반복문이 끝날 때까지 다른 스레드의 접근이 장시간 차단됨 (동시성 저하)

- 임계영역을 좁게 설정한 경우: thread_des()
 - 반복문 내부에서 매번 lock/unlock 을 호출하여 오버헤드가 급증함 (**실행 속도 매우 저하**)
 - 프로그램의 목적과 성능을 고려하여 임계 영역의 범위를 적절히 설정해야 함

```
$ gcc mutex.c -D_REENTRANT -o mutex -lpthread
$ ./mutex
result: 0
```

세마포어(Semaphore)

■ 세마포어의 생성과 소멸

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);
-> 성공 시 0, 실패 시 0 이외의 값 반환
```

- sem: 세마포어 생성시, 세마포어의 참조 값을 저장할 변수의 주소 전달
- pshared
 - 0 전달: 단일 프로세스 내 스레드 간 동기화
 - 0 이외의 값 전달: 다중 프로세스 간 공유 가능한 세마포어 생성
- value: 생성되는 세마포어의 초기 값 지정
 - 임계 영역에 동시 접근 가능한 스레드의 수를 제한할 수 있음
 - value=1: 한 번에 하나의 스레드만 임계 영역에 진입 (binary semaphore: mutex와 동일한 기능)

```
sem_t sem_one;
sem_init(&sem_one, 0, 1);
```

세마포어 획득과 반환

■ 세마포어의 획득과 반환

```
#include <semaphore.h>

int sem_post(sem_t *sem);
int sem_wait(sem_t *sem);
-> 성공 시 0, 실패 시 0 이외의 값 반환
```

- Binary Semaphore
 - 세마포어 카운트 == 0: 임계영역에 진입 불가
 - 세마포어 카운트 > 0: 임계영역에 진입 가능

■ 세마포어 동기화의 기본 구성

```
sem_wait(&sem); // 세마포어 값 1 감소
// 임계 영역의 시작
. . .
// 임계 영역의 끝
sem_post(&sem); // 세마포어 값 1 증가
```

- `sem_wait(&sem)`: semaphore 값을 1 감소시킴
 - if (세마포어 값 > 0), 값을 1 감소시킨 후 임계 영역 진입
 - if (세마포어 값 == 0), 값이 1 이상으로 증가할 때까지 대기
- `sem_post(&sem)`: semaphore 값을 1 증가시킴

- `sem_wait()`를 호출하고 임계영역에 진입한 스레드가 `sem_post()`를 호출하여 세마포어를 반환하기 전까지
 - 다른 스레드의 임계영역 진입을 차단함

Semaphore 이용한 동시 접근 제어: sem_one.c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
sem_t sem; // 세마포어 선언
```

```
void *thread_func1(void *arg)
{
```

```
    int thread_num = *(int *)arg;
```

```
    sem_wait(&sem); // 세마포어 대기
```

```
    /*----- 임계 영역 -----*/
```

```
    printf("Thread %d: Entered the critical section\n", thread_num);
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("[%d] count: %d\n", thread_num, i);
```

```
        sleep(1);
```

```
    }
```

```
    printf("Thread %d: Exit the critical section\n", thread_num);
```

```
    sem_post(&sem); // 세마포어 반환
```

```
    return NULL;
```

```
}
```

하나의 thread만 임계 영역에
접근하도록 제어 (동기화)

```
int main()
```

```
{
```

```
    pthread_t t1, t2;
```

```
    int num1 = 1, num2 = 2;
```

```
    sem_init(&sem, 0, 1); // 세마포어 초기값 1
```

```
    pthread_create(&t1, NULL, thread_func1, &num1);
```

```
    pthread_create(&t2, NULL, thread_func1, &num2);
```

```
    pthread_join(t1, NULL);
```

```
    pthread_join(t2, NULL);
```

```
    sem_destroy(&sem);
```

```
    return 0;
```

```
}
```

```
$ gcc sem_one.c -o sem_one -lpthread
$ ./sem_one
Thread 1: Entered the critical section
[1] count: 0
[1] count: 1
[1] count: 2
[1] count: 3
[1] count: 4
Thread 1: Exit the critical section
Thread 2: Entered the critical section
[2] count: 0
[2] count: 1
[2] count: 2
[2] count: 3
[2] count: 4
Thread 2: Exit the critical section
```

스레드 실행 순서 제어 예제 #1: 1개의 semaphore 사용

<sem_seq1.c>

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

sem_t sem; // 단일 세마포어 선언

void *thread1_func(void *arg)

```
{
    printf("Thread 1: Start work\n");
    sleep(1);
    printf("Thread 1: Finish work\n");
    sem_post(&sem); // sem값 1 증가
    return NULL;
}
```

void *thread2_func(void *arg)

```
{
    sem_wait(&sem);
    printf("Thread 2: Start work\n");
    sleep(1);
    printf("Thread 2: Finish work\n");
    return NULL;
}
```

③ semaphore
값 = 1

① 실행

②

① sem_post()를 대기



④

int main()

{

pthread_t t1, t2;

sem_init(&sem, 0, 0); // 초기값 0 (신호 대기 상태)

pthread_create(&t1, NULL, thread1_func, NULL);

pthread_create(&t2, NULL, thread2_func, NULL);

pthread_join(t1, NULL);

pthread_join(t2, NULL);

sem_destroy(&sem);

return 0;

}

\$ gcc sem_sequence.c -o sem_sequence -lpthread

\$./sem_sequence

Thread 1: Start work

Thread 1: Finish work

Thread 2: Start work

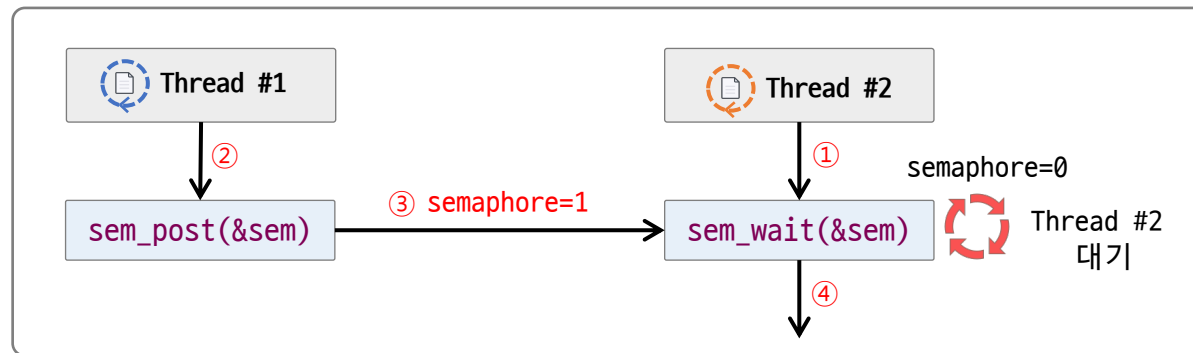
Thread 2: Finish work

항상 Thread1 실행 후
Thread2가 실행됨

Semaphore 활용한 멀티 스레드 실행 순서 제어

■ 실행 순서 제어 동작 원리

- 초기값(0 설정): `sem_init(&sem, 0, 0)`
 - 초기값이 0이므로, 먼저 `sem_wait(&sem)`을 호출한 스레드는 즉시 대기 상태(blocking)로 전환
 - 선행 스레드가 `sem_post(&sem)`를 호출하면 세마포어 값이 1로 증가하며 대기 중인 스레드를 깨움
- 실행 순서 제어



- 세마포어는 뮤텍스와 달리 소유권 개념이 없음

- Lock을 획득하지 않은 스레드도 `sem_post()`를 호출할 수 있음: 설계 시 실행 흐름이 꼬이지 않도록 주의

■ 활용 방안

활용 방안	세마포어 초기값	설명
실행 순서 제어(순차 실행)	0	후행 스레드가 선행 스레드의 작업 완료를 대기하도록 강제함
상호 배제(Mutual exclusion)	1	한 번에 하나의 스레드만 임계 영역에 접근 허용

스레드 실행 순서 제어 예제 #2: 1개의 semaphore 사용

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem; // 단일 세마포어 선언
void *thread1_func(void *arg)
{
    for (int i = 0; i < 5; i++) {
        sleep(1);
        printf("Thread 1: Finish work (i=%d)\n", i);
        sem_post(&sem); // Thread 2 실행 허용
    }
    return NULL;
}
```

sem_seq1.c와 차이점
- 여러 번 반복 수행

```
void *thread2_func(void *arg)
{
    for (int j = 0; j < 5; j++) {
        sem_wait(&sem); // Thread 1의 신호 대기
        sleep(1);
        printf("Thread 2: Finish work (j=%d)\n", j);
    }
    return NULL;
}
```

sem_seq1.c와 차이점
- 여러 번 반복 수행

```
int main()
{
    pthread_t t1, t2;

    sem_init(&sem, 0, 0); // 초기값 0 (신호 대기 상태)

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem);
    return 0;
}
```

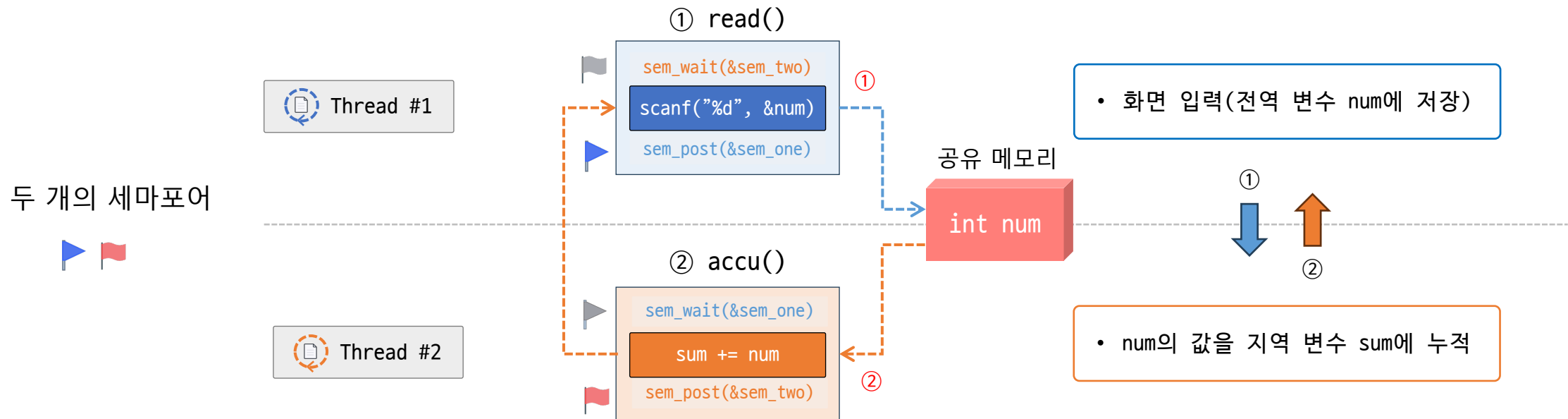
```
$ gcc sem_seq2.c -D_REENTRANT -o sem_seq2 -lpthread
$ ./sem_seq2
```

```
Thread 1: Finish work (i=0)
Thread 1: Finish work (i=1)
Thread 2: Finish work (j=0)
Thread 2: Finish work (j=1)
Thread 1: Finish work (i=2)
Thread 1: Finish work (i=3)
Thread 2: Finish work (j=2)
Thread 1: Finish work (i=4)
Thread 2: Finish work (j=3)
Thread 2: Finish work (j=4)
```

Thread1과 Thread2가 정확히
교대로 실행되지 않음

두 개의 semaphore를 활용한 접근 순서 동기화

- 1개 세마포어의 한계
 - 단일 세마포어로 2개의 스레드 함수를 제어할 경우
 - 한 번에 하나의 스레드만 접근하는 상호 배제(Mutual Exclusion)은 보장되지만,
 - 두 스레드 함수가 정확히 교대로 실행된다는 보장은 없음
- 해결책: 2개의 세마포어 사용
 - 2개의 세마포어를 교차 활용하여 스레드 간의 정확한 실행 순서를 동기화



두 개의 semaphore 사용: read()/accu() 순차 실행

Semaphore 초기값

`sem_one = 0`
`sem_two = 1`

먼저 실행하는
함수(read())에서 대기하는
sem_two를 초기값 1 설정

1. read()

```
sem_wait(&sem_two)
scanf("%d", &num)
sem_post(&sem_one)
```

② `sem_one=1`

2. accu()

```
sem_wait(&sem_one)
sum += num
sem_post(&sem_two)
```

④ `sem_two=1`

초기값 `sem_two=1`
Thread #1은 진입

선행 스레드

Thread #1

(1)

`sem_two(1->0 변경)`

① `scanf()` 수행 후

`sem_one(값 0->1로 증가)`

초기값 `sem_two=1`
Thread #1은 read() 진입

```
$ gcc semaphore.c -D_REENTRANT -o sem -lpthread
$ ./sem
Input num: 1
Input num: 2
Input num: 3
Input num: 4
Input num: 5
Result: 15
```

read() -> accu()

초기값 `sem_one=0`
Thread #2은 대기(blocking)

후행 스레드

Thread #2

(0)

`sem_one(1->0 변경)`

③ sum에 누적

`sem_two(값 0->1로 증가)`

선행 스레드의 read()함수에서
`sem_post(&sem_one)`를 호출하면, 대기 중이던
후행 스레드가 accu() 함수 내부로 진입할 수
있음

두 개의 semaphore를 활용한 접근 순서 동기화: semaphore.c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
```

```
void *read(void *arg);
void *accu(void *arg);
static sem_t sem_one;
static sem_t sem_two;
static int num;
```

static 전역 변수
- 선언된 파일 내부에서만 사용 가능

```
int main(int argc, char *argv[])
{
```

```
    pthread_t id_t1, id_t2;
    sem_init(&sem_one, 0, 0);
    sem_init(&sem_two, 0, 1);
```

sem_one 초기값=0,
sem_two 초기값=1
read() 함수가 먼저 수행됨

```
    pthread_create(&id_t1, NULL, read, NULL);
    pthread_create(&id_t2, NULL, accu, NULL);
```

```
    pthread_join(id_t1, NULL);
    pthread_join(id_t2, NULL);
```

```
    sem_destroy(&sem_one);
    sem_destroy(&sem_two);
    return 0;
```

```
}
```

sem_two = 1

```
void *read(void *arg)
```

```
{
    int i;
    for(i=0; i<5; i++)
    {
```

```
        fputs("Input num: ", stdout);
```

```
        sem_wait(&sem_two);
```

```
        scanf("%d", &num);
```

```
        sem_post(&sem_one);
```

```
    }
```

```
    return NULL;
```

```
}
```

```
void *accu(void *arg)
```

```
{
```

```
    int sum=0, i;
```

```
    for(i=0; i<5; i++)
```

```
    {
```

```
        sem_wait(&sem_one);
```

```
        sum += num;
```

```
        sem_post(&sem_two);
```

```
        printf("sum: %d\n", sum);
```

```
    }
```

```
    printf("Result: %d\n", sum);
```

```
    return NULL;
```

```
}
```

sem_one = 0

두 개의 세마포어를 이용하여
접근 순서를 동기화

sem_two--

sem_one++

sem_one--

sem_two++

스레드의 종료 대기 및 소멸

■ pthread_detach()

- 스레드에게 할당된 메모리 자원을 자동으로 운영체제에 반환
 - 별도로 pthread_join()을 호출할 필요 없음

```
#include <pthread.h>
```

```
int pthread_detach(pthread_t thread);
```

-> 성공 시 0, 실패 시 0 이외의 값 반환

➤ thread: 종료와 동시에 소멸시킬 스레드의 ID

■ 스레드 소멸의 필요성

- 스레드 함수가 종료되어 값을 반환(return)하더라도 메모리 자원은 자동으로 해제되지 않음
- 따라서 다음 두 함수 중 하나를 호출해서 스레드의 소멸 및 자원 해제를 명시적으로 수행해야 함

- pthread_join(): 호출한 스레드를 대기 상태(blocking)로 만들며 종료 시 자원 해제
- pthread_detach(): 대기 상태 없이(non-blocking) 대상 스레드 종료와 동시에 자원 해제

멀티스레드 기반의 다중 접속 서버 구현: 클라이언트 접속

■ chat_server.c 일부

```
while(1)
{
    clnt_adr_sz=sizeof(clnt_adr);
    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr,&clnt_adr_sz);

    pthread_mutex_lock(&mutx);
    clnt_socks[clnt_cnt++]=clnt_sock;
    pthread_mutex_unlock(&mutx);

    pthread_create(&t_id, NULL, handle_clnt, (void*)&clnt_sock);
    pthread_detach(t_id);

    printf("Connected client IP: %s \n", inet_ntoa(clnt_adr.sin_addr));
}
```

• 새로 접속한 클라이언트의 소켓 정보를 전역 배열(clnt_socks[])에 추가
- 배열 동시 접근 충돌을 막기 위해 pthread_mutex_lock()/unlock() 사용

• 스레드에 소켓 디스크립터(clnt_sock) 전달

- 클라이언트의 연결 수락(accept)시, 서버는 해당 클라이언트와 통신할 새로운 스레드를 생성 및 생성된 스레드에게 소켓 디스크립터(clnt_sock)를 전달
- 생성된 스레드는 handle_clnt() 함수를 실행하여 독립적으로 클라이언트에게 서비스를 제공
- 서비스 완료 후 스레드 종료 시 스레드의 자원 해제를 위해 pthread_detach() 호출

멀티스레드 기반의 다중 접속 서버 구현: 서비스 제공

```
void *handle_clnt(void * arg)
{
    int clnt_sock = *((int*)arg);
    int str_len=0, i;
    char msg[BUF_SIZE];

    while((str_len=read(clnt_sock, msg, sizeof(msg)))!=0)
        send_msg(msg, str_len);

    pthread_mutex_lock(&mutex);
    for(i=0; i<clnt_cnt; i++) // remove disconnected client
    {
        printf("clnt_sock: %d, clnt_cnt: %d\n", clnt_sock, clnt_cnt);
        if(clnt_sock==clnt_socks[i])
        {
            while(i < clnt_cnt)
            {
                clnt_socks[i]=clnt_socks[i+1];
                i++; // 클라이언트 종료 시 무한루프 발생 문제점 수정
            }
            break;
        }
    }
    clnt_cnt--;
    pthread_mutex_unlock(&mutex);
    close(clnt_sock);
    return NULL;
}
```

클라이언트 종료

소스 수정 부분
(연결 해제한 클라이언트 삭제)

```
void send_msg(char * msg, int len) // send to all
{
    int i;
    pthread_mutex_lock(&mutex);

    for(i=0; i<clnt_cnt; i++)
        write(clnt_socks[i], msg, len);

    pthread_mutex_unlock(&mutex);
}
```

수신된 데이터를 모든 클라이언트에게 전송

■ chat_server.c 일부

- -클라이언트에게 서비스 제공 부분

- 수신된 메시지를 모든 클라이언트에게 전송
- 클라이언트의 소켓 정보를 참조하는 부분에 동기화 코드
- 클라이언트의 소켓 정보를 참조하는 동안 소켓의 추가 및 삭제(종료)를 막기 위함

chat_server.c 전체 소스 #1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <pthread.h>

#define BUF_SIZE 100
#define MAX_CLNT 256
void * handle_clnt(void * arg);
void send_msg(char * msg, int len);
void error_handling(char * msg);

int clnt_cnt=0;
int clnt_socks[MAX_CLNT];
pthread_mutex_t mutex;

int main(int argc, char *argv[])
{
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_addr, clnt_addr;
    socklen_t clnt_addr_sz;
    pthread_t t_id;
    if(argc!=2) {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }
}
```

```
pthread_mutex_init(&mutex, NULL);
serv_sock=socket(PF_INET, SOCK_STREAM, 0);

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr=htonl(INADDR_ANY);
serv_addr.sin_port=htons(atoi(argv[1]));
if(bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr))==-1)
    error_handling("bind() error");
if(listen(serv_sock, 5)==-1)
    error_handling("listen() error");
while(1)
{
    clnt_addr_sz=sizeof(clnt_addr);
    clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_sz);

    pthread_mutex_lock(&mutex);
    clnt_socks[clnt_cnt++]=clnt_sock;
    pthread_mutex_unlock(&mutex);

    pthread_create(&t_id, NULL, handle_clnt, (void*)&clnt_sock);
    pthread_detach(t_id);

    printf("Connected client IP: %s \n", inet_ntoa(clnt_addr.sin_addr));
}
close(serv_sock);
return 0;
}
```

동기화 코드

chat_server.c 전체 소스 #2

```
void *handle_clnt(void * arg)
{
    int clnt_sock=*((int*)arg);
    int str_len=0, i;
    char msg[BUF_SIZE];
    while((str_len=read(clnt_sock, msg, sizeof(msg)))!=0)
        send_msg(msg, str_len);

    pthread_mutex_lock(&mutx);
    for(i=0; i<clnt_cnt; i++) // remove disconnected client
    {
        if(clnt_sock==clnt_socks[i])
        {
            while(i < clnt_cnt)
            {
                clnt_socks[i]=clnt_socks[i+1];
                i++;
            }
            break;
        }
    }
    clnt_cnt--;
    pthread_mutex_unlock(&mutx);
    close(clnt_sock);
    return NULL;
}
```

클라이언트와 통신

소스 수정 부분
(연결 해제한 클라이언트 삭제)

```
void send_msg(char * msg, int len) // send to all
{
    int i;
    pthread_mutex_lock(&mutx);
    for(i=0; i<clnt_cnt; i++)
        write(clnt_socks[i], msg, len);
    pthread_mutex_unlock(&mutx);
}

void error_handling(char * msg)
{
    fputs(msg, stderr);
    fputc('\n', stderr);
    exit(1);
}
```

동기화 코드:
클라이언트 정보에 접근

chat_clnt.c 전체 소스 #1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>
#define BUF_SIZE 100
#define NAME_SIZE 20

void *send_msg(void * arg);
void *recv_msg(void * arg);
void error_handling(char * msg);

char name[NAME_SIZE]="[DEFAULT]";
char msg[BUF_SIZE];

int main(int argc, char *argv[])
{
    int sock;
    struct sockaddr_in serv_addr;
    pthread_t snd_thread, rcv_thread;
    void * thread_return;
```

```
if(argc!=4) {
    printf("Usage : %s <IP> <port> <name>\n", argv[0]);
    exit(1);
}
sprintf(name, "[%s]", argv[3]);

sock=socket(PF_INET, SOCK_STREAM, 0);
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr=inet_addr(argv[1]);
serv_addr.sin_port=htons(atoi(argv[2]));

if(connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr))==-1)
    error_handling("connect() error");
```

- 데이터 송신용 스레드: snd_thread 생성
- 데이터 수신용 스레드: rcv_thread 생성

```
pthread_create(&snd_thread, NULL, send_msg, (void*)&sock);
pthread_create(&rcv_thread, NULL, recv_msg, (void*)&sock);
```

```
pthread_join(snd_thread, &thread_return);
pthread_join(rcv_thread, &thread_return);
```

```
close(sock);
return 0;
```

```
}
```


chat_clnt.c 전체 소스 #2

```
void *send_msg(void * arg) // send thread main
{
    int sock = *((int*)arg);
    char name_msg[NAME_SIZE+BUF_SIZE];

    while(1)
    {
        fgets(msg, BUF_SIZE, stdin);
        if(!strcmp(msg, "q\n") || !strcmp(msg, "Q\n"))
        {
            close(sock);
            exit(0);
        }
        sprintf(name_msg, "%s %s", name, msg);

        write(sock, name_msg, strlen(name_msg));
    }
    return NULL;
}
```

```
void *recv_msg(void * arg) // read thread main
{
    int sock = *((int*)arg);
    char name_msg[NAME_SIZE+BUF_SIZE];
    int str_len;

    while(1)
    {
        str_len=read(sock, name_msg, NAME_SIZE+BUF_SIZE-1);
        if(str_len==-1)
            return (void*)-1;
        name_msg[str_len]=0;
        fputs(name_msg, stdout);
    }
    return NULL;
}

void error_handling(char *msg)
{
    fputs(msg, stderr);
    fputc('\n', stderr);
    exit(1);
}
```

실행

서버 실행

```
$ gcc chat_server.c -D_REENTRANT -o cserv -lpthread
$ gcc chat_clnt.c -D_REENTRANT -o cclnt -lpthread

$ ./cserv 9190
Connected client IP: 127.0.0.1
Connected client IP: 127.0.0.1
Connected client IP: 127.0.0.1
```

클라이언트 #1 실행

```
$ ./cclnt 127.0.0.1 9190 Alice
[Bob] Hi
Hi
[Alice] Hi
[Charlie] Hello
```

클라이언트 #2 실행

```
$ ./cclnt 127.0.0.1 9190 Bob
Hi
[Bob] Hi
[Alice] Hi
[Charlie] Hello
```

클라이언트 #3 실행

```
$ ./cclnt 127.0.0.1 9190 Charlie
[Bob] Hi
[Alice] Hi
Hello
[Charlie] Hello
```

Questions?

LMS Q&A 게시판에 올려주세요.